

Application Security Course Project: Automated Assessment of VulnWeb Targets

Prepared by: Ria Mary Abraham

Course : Application Security

Group No: 5

Table of Contents

1. Executive Summary
2. Scope and Methodology
3. Findings Summary and Impact
4. Detailed Findings and Impact
5. Mitigation Strategies
6. Testing Notes
7. Conclusion
8. Appendix: References and Tools

1. Executive Summary

A security review was conducted on the VulnWeb test applications (testasp, testaspnet, testphp, and rest) primarily utilizing the automated scanning capabilities of OWASP ZAP (v2.16.1). The scan and subsequent manual analysis identified a total of 21 unique security alerts, highlighting both major application-level flaws and extensive configuration issues.

The overall security posture is deemed

critically weak. Confirmed fundamental application vulnerabilities include:

- SQL Injection (SQLi) on the testasp and testphp sites.
- Reflected Cross-Site Scripting (XSS) on testaspnet.
- Insecure File Upload on testphp.

These vulnerabilities could enable data theft, modification, or even

Remote Code Execution (RCE) in the case of Insecure File Upload. In addition to these critical application flaws, a High-risk Vulnerable JS Library was found on <http://rest.vulnweb.cm/>. The automated scan also flagged systemic, Medium-risk configuration problems across all applications, specifically the Absence of Anti-CSRF Tokens, Missing Anti-clickjacking Header, and Content Security Policy (CSP) Header Not Set.

Immediate remediation is strongly recommended, prioritizing the SQL Injection and Insecure File Upload flaws, patching the vulnerable JavaScript library, and implementing all missing security headers and tokens to establish a base layer of defense. A follow-up verification test is necessary once fixes are applied

2. Scope and Methodology

Scope and Objectives

The assessment covered the following four hosts, as defined by the ZAP report parameters:

<http://rest.vulnweb.com>

http://testphp.vulnweb.com

http://testasp.vulnweb.com

http://testaspnet.vulnweb.com

The primary objective was to use an automated scanner (OWASP ZAP) to discover common web application and configuration vulnerabilities , document the results, and propose mitigation strategies.

Methodology and Tools

The review employed

Automated Scanning across all four targets using OWASP ZAP's Active Scan capability (Version 2.16.1). The final report included alerts from all risk levels (High, Medium, Low, and Informational), excluding those with a "False Positive" confidence level

3. Findings Summary and Impact

Key Vulnerabilities (High-Impact)

vulnerability	Affected hosts	severity
SQL Injection (SQLi)	testasp.vulnweb.com, testphp.vulnweb.com	High
Insecure File Upload	testphp.vulnweb.com	High
Vulnerable JS Library	http://rest.vulnweb.com	High
Reflected Cross-Site Scripting (XSS)	testaspnet.vulnweb.com	Medium
Absence of Anti-CSRF Tokens	testasp.vulnweb.com, testaspnet.vulnweb.com	Medium
Missing Anti-clickjacking Header	All sites	Medium

Target Application Summary

The following table summarizes the High and Medium-risk alerts noted per host:

Host	Description/Technology	Alerts Noted(Risk>=Medium
http://rest.vulnweb.com	RESTful API demonstration site.	1High, 2 Medium
http://testphp.vulnweb.com	PHP-based demo site .	0High, 0Medium
http://testaspnet.vulnweb.com	Classic ASP test site .	0High, 0Medium
http://testaspnet.vulnweb.com	ASP.NET sample site.	0High, 3Medium

4. Detailed Findings and Impact

SQL Injection (SQLi)

Affected Hosts: testasp.vulnweb.com, testphp.vulnweb.com.

Proof-of-Concept (PoC) Payload Example: 'OR '1'='1.

Impact: Enables database enumeration, data theft, and potential shell access.

Insecure File Upload

Affected Host: http://testphp.vulnweb.com/.

Proof-of-Concept (PoC): Uploading a malicious shell file (e.g., cmd.php containing `<?php system($_GET['cmd']); ?>`) and executing it via its public URL.

Impact: Potential for complete system/server compromise and Remote Code Execution.

Reflected Cross-Site Scripting (XSS)

Affected Host: testaspnet.vulnweb.com.

Proof-of-Concept (PoC) URL Example:

[http://testaspnet.vulnweb.com/search.aspx?q=<script>alert\('XSS'\)</script>](http://testaspnet.vulnweb.com/search.aspx?q=<script>alert('XSS')</script>).

Impact: Allows session hijacking, cookie theft, and execution of arbitrary code within a user's browser

ZAP REPORT



Summaries

Alert counts by risk and confidence

This table shows the number of alerts for each level of risk and confidence included in the report.

(The percentages in brackets represent the count as a percentage of the total number of alerts included in the report, rounded to one decimal place.)

		Confidence			
Risk		User Confirmed	High	Medium	Low
	High	0 (0.0%)	0 (0.0%)	1 (4.8%)	0 (0.0%)
	Medium	0 (0.0%)	2 (9.5%)	2 (9.5%)	1 (4.8%)
	Low	0 (0.0%)	2 (9.5%)	6 (28.6%)	0 (0.0%)
	Informational	0 (0.0%)	1 (4.8%)	4 (19.0%)	2 (9.5%)
	Total	0 (0.0%)	5 (23.8%)	13 (61.9%)	3 (14.3%)
		Total			
		0 (0.0%)	5 (23.8%)	13 (61.9%)	3 (14.3%)

Alert counts by site and risk

Alert counts by site and risk

This table shows, for each site for which one or more alerts were raised, the number of alerts raised at each risk level.

Alerts with a confidence level of "False Positive" have been excluded from these counts.

(The numbers in brackets are the number of alerts raised for the site at or above that risk level.)

		Risk			
Site		High (= High)	Medium (>= Medium)	Low (>= Low)	Informational (>= Informational)
	http://rest.vulnweb.com	1 (1)	2 (3)	0 (3)	0 (3)
	http://testphp.vulnweb.com	0 (0)	0 (0)	0 (0)	1 (1)
	http://testasp.vulnweb.com	0 (0)	0 (0)	3 (3)	0 (3)
	http://testaspnet.vulnweb.com	0 (0)	3 (3)	5 (8)	6 (14)
	Total	1 (1)	5 (8)	8 (14)	7 (14)

Alert counts by alert type

This table shows the number of alerts of each alert type, together with the alert type's risk level.

Alert counts by alert type

This table shows the number of alerts of each alert type, together with the alert type's risk level.

(The percentages in brackets represent each count as a percentage, rounded to one decimal place, of the total number of alerts included in this report.)

Alert type	Risk	Count
Vulnerable JS Library	High	1 (4.8%)
Absence of Anti-CSRF Tokens	Medium	95 (452.4%)
Content Security Policy (CSP) Header Not Set	Medium	156 (742.9%)
Hidden File Found	Medium	1 (4.8%)
Missing Anti-clickjacking Header	Medium	119 (566.7%)
Vulnerable JS Library	Medium	2 (9.5%)
Application Error Disclosure	Low	14 (66.7%)
Cookie No HttpOnly Flag	Low	1 (4.8%)

Cookie No HttpOnly Flag	Low	1 (4.8%)
Cookie without SameSite Attribute	Low	2 (9.5%)
Information Disclosure - Debug Error Messages	Low	14 (66.7%)
Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	Low	225 (1,071.4%)
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	273 (1,308.0%)
X-AspNet-Version Response Header	Low	28 (133.3%)
X-Content-Type-Options Header Missing	Low	202 (961.9%)
Authentication Request Identified	Informational	3 (14.3%)
Charset Mismatch (Header Versus Meta Content-Type Charset)	Informational	34 (161.9%)
Information Disclosure - Suspicious Comments	Informational	8 (38.1%)
Modern Web Application	Informational	10

Modern Web Application	Informational	10 (47.6%)
Session Management Response Identified	Informational	4 (19.0%)
User Agent Fuzzer	Informational	12 (57.1%)
User Controllable HTML Element Attribute (Potential XSS)	Informational	49 (233.3%)
Total		21

Alerts

Risk=High, Confidence=Medium (1)

http://rest.vulnweb.com (1)

Vulnerable JS Library (1)

► GET http://rest.vulnweb.com/docs/vendor/handlebars.min.js?v=1758811348061

Risk=Medium, Confidence=High (2)

5. Mitigation Strategies

Critical Priority (High Risk)

- Patch Vulnerable Libraries (F-03): Immediately identify the version of Handlebars.js and any other vulnerable JS libraries on http://rest.vulnweb.com/ and update them to a secure, patched version. If an update is impossible, the library must be removed, or a WAF/virtual patching should be applied to mitigate known CVEs.

High Priority (Medium Risk)

- Implement Anti-Clickjacking (F-06): Deploy either the HTTP header X-Frame-Options: DENY (or SAMEORIGIN) or the CSP directive Content-Security-Policy: frame-ancestors 'none'; to prevent the site from being framed.

- Set Strong CSP (Systemic): Deploy a restrictive Content-Security-Policy header. This whitelist of trusted sources for scripts and content will act as a secondary defense layer against XSS and injection attacks.
- Implement Anti-CSRF Tokens (F-05): Use Synchronizer Tokens for all state-changing requests (POST, PUT, DELETE) to protect against Cross-Site Request Forgery (CSRF) attacks.

Low Priority (Low Risk)

- Suppress Information Disclosure: Configure web servers to remove or suppress identifying headers like Server, X-Powered-By, and X-AspNet-Version.
- Set Cookie Flags: Ensure all cookies are set with the HttpOnly flag to prevent client-side script access, which reduces the impact of XSS. Also, use the Same Site attribute (Strict/Lax) to mitigate CSRF.
- Prevent MIME-Type Sniffing: Implement the X-Content-Type-Options: nosniff header

6. Testing Notes

Assessment Limitations

This assessment was conducted primarily using an automated scanner. Notable limitations include:

- Common, manually-testable critical flaws (like SQLi) were found via manual analysis, not flagged by the scanner in its High/Medium categories, suggesting a reliance on manual verification is key.
- Many Medium-risk alerts, such as the Absence of Anti-CSRF Tokens (F-05), were assigned a Low Confidence by the scanner, requiring manual confirmation.
- The large count of Low-risk alerts (e.g., 1300.0% for Server Leaks) indicates systemic misconfiguration issues across the entire scope.

7. Conclusion

The security posture of the VulnWeb test applications is characterized by a high-risk dependency vulnerability on <http://rest.vulnweb.com> and a systemic failure to implement foundational security defenses across all sites. The overwhelming presence of missing headers and tokens confirms that developers must focus on secure coding fundamentals and hardening server configurations. The immediate priorities are patching vulnerable libraries and implementing robust security headers and CSRF defenses.

8. Appendix: References and Tools

Key References

This assessment utilized resources from the Open Web Application Security Project (OWASP):

OWASP Top Ten: General guidance on critical web application security risks.

OWASP Content Security Policy (CSP) Cheat Sheet: Guidance on implementing the CSP header (Mitigation for F-06).

OWASP Clickjacking Defense Cheat Sheet: Guidance on implementing frame-ancestors and X-Frame-Options (Mitigation for F-06).

OWASP CSRF Prevention Cheat Sheet: Guidance on implementing Synchronizer Tokens (Mitigation for F-05).

Common Vulnerabilities and Exposures (CVE) Database: Used for researching specific vulnerabilities in JS Libraries (F-03).

ZAP Scan Details

Scanner Used: OWASP ZAP (Version 2.16.1).

Sites Included: The scan included all four target applications.

Risk/Confidence Levels: The report includes High, Medium, Low, and Informational risk levels, and User Confirmed, High, Medium, and Low confidence levels